

FRONTEND • BACKEND • DATABASE

FRONTEND — Web / PWA

HTML · CSS · JavaScript | folder web/

- Farmer screens: Photo, Field, News, Shop, Call, Help, chat
- Camera / gallery = HTML file input (no Capacitor Camera plugin)
- Bangla / English, login, vendor desk, admin desk
- PWA service worker `sw.js`

BACKEND / API — FastAPI

`backend/main.py` · Uvicorn. Frontend calls:

- `POST /api/predict` — photo → leaf → crop → disease
- `POST /api/chat`, `/chat/vision`, `/chat/stt`
- Farmer auth `/api/auth/*` · vendor `/api/vendor/*`
- Admin `/api/admin/*` · shop, news, advice, farm APIs

DATA / STORAGE

Server-side application data



- `users.db` — farmer accounts
- `agroscan_shop.db` — shop & vendors
- `diseases/` — 1 JSON file per disease (136)
- `*.pt` — PyTorch CNN weights

DATABASE NOTE

The backend DBMS is **SQLite**. Farmer sessions: `data/users.db`. Shop / vendor / admin: `data/agroscan_shop.db`.

The frontend also uses browser **localStorage** for cart, scan history, field plots, language, and the session token. That is not the server database.

CNN weights are files, not rows: `leaf_gate.pt`, `leaf_type.pt`, `disease_*.pt`.

Live: agroscan.mujahidulhaquejihad.com

MOBILE APP — Capacitor Android

Capacitor 6.2 | `com.mujahidulhaquejihad.agroscan`

- Same web UI copied into `mobile/www`
- No on-device model — scan, chat, shop call the hosted API
- Native plugins: Geolocation, App, Status Bar
- APK built with Gradle; API base = production URL

AI + APPLICATION SERVICES

Runs on the server, only through those APIs

- L1 MobileNetV3-Large → L2 EfficientNet-B3 → L3 ensemble (B3 + ResNet-50 + DenseNet-121), crop-masked, highest confidence wins
- Advice comes from the JSON pack, not from the CNN
- Chat: clarify → pack → KB → Gemini + TF-IDF RAG ($k = 4$)
- Shop orders, vendor stock, admin users
- Weather is not this API — the page calls Open-Meteo in the browser

